

Module 4 Structured Note: Validating a String

1. What this module is about

Module 4 teaches how to **validate a string** in C/C++.

Validation means:

Checking whether a string follows a rule.

In this module, the rule is simple:

A string is valid only if it contains letters and numbers.

So the string may contain:

- Uppercase letters: `A` to `Z`
- Lowercase letters: `a` to `z`
- Digits: `0` to `9`

The string must not contain special symbols like:

`? ! @ # _ space`

unless we specifically change the rule to allow them.

2. Required idea from previous modules

Before Module 4, remember this important pattern:

```
for (i = 0; name[i] != '\0'; i++) {  
    // check name[i]  
}
```

This means:

1. Start from index `0`.
2. Check one character at a time.
3. Keep moving forward.
4. Stop when you reach `\0`.

The null terminator `\0` tells us where the string ends.

Example:

```
char name[] = "Anil2321";
```

Memory view:

```
A n i l 2 3 2 1 \0
```

The loop checks:

```
A, n, i, l, 2, 3, 2, 1
```

Then it stops at:

```
\0
```

Part A: Meaning of string validation

3. What does validation mean?

Validation means checking whether input is acceptable.

For example, suppose we are checking a username.

Rule:

```
Only letters and numbers are allowed.
```

Valid examples:

```
Anil2321  
User123  
abc99  
A1B2C3
```

Invalid examples:

```
An?1
user_name
Hello!
my name
```

Why are these invalid?

String	Problem
An?1	contains ?
user_name	contains _
Hello!	contains !
my name	contains a space

The original Module 4 rule allows only letters and digits.

4. Main goal of validation

The goal is to check every character.

For each character:

```
If it is an uppercase letter, it is allowed.
If it is a lowercase letter, it is allowed.
If it is a digit, it is allowed.
Otherwise, it is invalid.
```

If even one character is invalid, the whole string is invalid.

Example:

```
An?1
```

A is valid.

n is valid.

? is invalid.

So the full string is invalid.

We do not need to check `1` after finding `?`, because the answer is already known.

Part B: Allowed ASCII ranges

5. Uppercase letter range

Uppercase letters are from `A` to `Z`.

ASCII range:

```
A to Z = 65 to 90
```

In C, we can check uppercase like this:

```
name[i] >= 65 && name[i] <= 90
```

A more readable version is:

```
name[i] >= 'A' && name[i] <= 'Z'
```

Both mean the same thing.

Recommended beginner version:

```
name[i] >= 'A' && name[i] <= 'Z'
```

This is easier to understand because it clearly shows that we are checking letters.

6. Lowercase letter range

Lowercase letters are from `a` to `z`.

ASCII range:

```
a to z = 97 to 122
```

In C, we can check lowercase like this:

```
name[i] >= 97 && name[i] <= 122
```

A more readable version is:

```
name[i] >= 'a' && name[i] <= 'z'
```

7. Digit range

Digits are from 0 to 9.

ASCII range:

```
0 to 9 = 48 to 57
```

Important:

The character '0' is not the same as the number 0.

```
'0' // character zero, ASCII value 48  
0  // number zero
```

To check whether a character is a digit:

```
name[i] >= '0' && name[i] <= '9'
```

This checks characters like:

```
'0', '1', '2', ..., '9'
```

8. Allowed safe zones

The three allowed safe zones are:

```
'A' to 'Z'  
'a' to 'z'  
'0' to '9'
```

Or using ASCII numbers:

```
65 to 90  
97 to 122  
48 to 57
```

So a character is valid if it belongs to at least one of these ranges.

Part C: The validation logic

9. Simple logic in words

For each character in the string:

```
If character is uppercase:  
    valid, continue  
Else if character is lowercase:  
    valid, continue  
Else if character is digit:  
    valid, continue  
Else:  
    invalid, return 0
```

At the end:

```
If no invalid character was found, return 1.
```

10. Return values

The validation function returns an integer.

Common convention:

```
1 means true / valid  
0 means false / invalid
```

So:

```
return 1;
```

means:

```
The string is valid.
```

And:

```
return 0;
```

means:

```
The string is invalid.
```

11. Early return

Module 4 uses an important idea called **early return**.

Early return means:

As soon as you find an invalid character, stop the function immediately.

Example:

```
An?1
```

Once the function sees `?`, it immediately returns `0`.

It does not need to check the remaining characters.

Why?

Because one invalid character is enough to make the whole string invalid.

Part D: Code for string validation

12. Main code pattern

```
#include <stdio.h>

int validate(char *name) {
    int i;

    for (i = 0; name[i] != '\0'; i++) {
        if (!(name[i] >= 'A' && name[i] <= 'Z') &&
            !(name[i] >= 'a' && name[i] <= 'z') &&
            !(name[i] >= '0' && name[i] <= '9')) {
            return 0;
        }
    }

    return 1;
}

int main() {
    char name[] = "An?1";

    if (validate(name)) {
        printf("Valid String\n");
    } else {
        printf("Invalid String\n");
    }

    return 0;
}
```

Output:

```
Invalid String
```

13. Understanding `char *name`

The function is written as:


```
int validate(char *name)
```

This means `name` points to the first character of the string.

So if the string is:

```
An?1
```

Then `name` lets us access characters like this:

```
name[0] // A
name[1] // n
name[2] // ?
name[3] // 1
name[4] // \0
```

For beginners, you can think of:

```
char *name
```

as:

```
The string passed into the function.
```

14. Understanding the loop

```
for (i = 0; name[i] != '\0'; i++)
```

This loop means:

```
Start at index 0.
Continue while the current character is not \0.
Move to the next character after each check.
```

It stops at the end of the string.

15. Understanding the `if` condition

The condition is:

```
if (!(name[i] >= 'A' && name[i] <= 'Z') &&  
    !(name[i] >= 'a' && name[i] <= 'z') &&  
    !(name[i] >= '0' && name[i] <= '9'))
```

This means:

If the character is NOT uppercase
AND the character is NOT lowercase
AND the character is NOT a digit,
then the character is invalid.

This is called **negative logic**.

Instead of saying:

If it is allowed, continue.

we say:

If it is not in any allowed range, reject it.

16. Why we use `&&`

The condition uses `&&`, which means **AND**.

```
!uppercase && !lowercase && !digit
```

This means:

The character must fail all three checks to be invalid.

Example: `?`

```
Is ? uppercase? No.  
Is ? lowercase? No.  
Is ? digit? No.
```

So:

```
? is invalid.
```

Example: `A`

```
Is A uppercase? Yes.
```

So `A` is valid.

It does not matter that `A` is not lowercase and not a digit, because it already passed one allowed range.

Part E: Dry runs

17. Dry run 1: Invalid string `"An?1"`

String:

```
A n ? 1 \0
```

Initial value:

```
i = 0
```

Trace:

i	Character	Uppercase?	Lowercase?	Digit?	Action
0	<code>A</code>	Yes	No	No	valid, continue
1	<code>n</code>	No	Yes	No	valid, continue
2	<code>?</code>	No	No	No	invalid, return <code>0</code>
3	<code>1</code>	not checked	not checked	not checked	already stopped

Final result:

Invalid String

Reason:

? is not a letter and not a digit.

18. Dry run 2: Valid string "Anil2321"

String:

A n i l 2 3 2 1 \0

Trace:

i	Character	Type	Action
0	A	uppercase	valid
1	n	lowercase	valid
2	i	lowercase	valid
3	l	lowercase	valid
4	2	digit	valid
5	3	digit	valid
6	2	digit	valid
7	1	digit	valid
8	\0	end of string	stop loop

Since no invalid character was found, the function reaches:

```
return 1;
```

Final result:

Valid String

19. Dry run 3: Invalid string "user_name"

String:

u s e r _ n a m e \0

Trace:

i	Character	Type	Action
0	u	lowercase	valid
1	s	lowercase	valid
2	e	lowercase	valid
3	r	lowercase	valid
4	_	symbol	invalid, return 0

Final result:

Invalid String

Reason:

_ is not allowed by the original rule.

Part F: Cleaner beginner version

20. Version using helper variables

The original condition is correct, but it can look difficult.

A cleaner beginner-friendly version uses helper variables:

```

#include <stdio.h>

int validate(char *name) {
    int i;

    for (i = 0; name[i] != '\0'; i++) {
        int isUpper = name[i] >= 'A' && name[i] <= 'Z';
        int isLower = name[i] >= 'a' && name[i] <= 'z';
        int isDigit = name[i] >= '0' && name[i] <= '9';

        if (!isUpper && !isLower && !isDigit) {
            return 0;
        }
    }

    return 1;
}

int main() {
    char name[] = "Anil2321";

    if (validate(name)) {
        printf("Valid String\n");
    } else {
        printf("Invalid String\n");
    }

    return 0;
}

```

This version says:

Check whether the character is uppercase.
 Check whether the character is lowercase.
 Check whether the character is a digit.
 If it is none of these, reject it.

Same logic, easier to read.

Part G: Important rule about returning 1

21. Do not return 1 too early

This is a common mistake:

```
int validate(char *name) {
    int i;

    for (i = 0; name[i] != '\0'; i++) {
        if (name[i] >= 'A' && name[i] <= 'Z') {
            return 1;
        }
    }

    return 0;
}
```

Why is this wrong?

Because it returns 1 after finding only one valid character.

Example:

A???

The first character A is valid.

But the full string is not valid.

The function must check every character before saying the string is valid.

Correct rule:

Return 0 immediately when you find an invalid character.
Return 1 only after the full string has passed.

Part H: Modifying the validation rule

22. Allowing underscore

Original rule:

Only letters and digits are allowed.

So this is invalid:

user_name

because it contains .

If we want to allow , we add it as another safe condition.

```
int validate(char *name) {
    int i;

    for (i = 0; name[i] != '\0'; i++) {
        if (!(name[i] >= 'A' && name[i] <= 'Z') &&
            !(name[i] >= 'a' && name[i] <= 'z') &&
            !(name[i] >= '0' && name[i] <= '9') &&
            !(name[i] == '_')) {
            return 0;
        }
    }

    return 1;
}
```

Now this is valid:

user_name

23. Allowing spaces

Original rule rejects spaces.

Example:

```
my name
```

This is invalid because it contains a space.

If we want to allow spaces, we add:

```
name[i] == ' '
```

Full condition:

```
if (!(name[i] >= 'A' && name[i] <= 'Z') &&
    !(name[i] >= 'a' && name[i] <= 'z') &&
    !(name[i] >= '0' && name[i] <= '9') &&
    !(name[i] == ' ')) {
    return 0;
}
```

Now spaces are allowed.

But for usernames, spaces are usually rejected because usernames are easier to handle without spaces.

24. Allowing both underscore and spaces

If the rule says letters, digits, underscores, and spaces are allowed:

```
if (!(name[i] >= 'A' && name[i] <= 'Z') &&
    !(name[i] >= 'a' && name[i] <= 'z') &&
    !(name[i] >= '0' && name[i] <= '9') &&
    !(name[i] == '_') &&
    !(name[i] == ' ')) {
    return 0;
}
```

Now these are valid:

```
user_name  
my name  
User_123
```

The important idea is:

Every allowed character must be added as a safe zone or safe condition.

Part I: Test cases

25. Test 1: Valid letters and digits

Input:

```
Anil2321
```

Expected output:

```
Valid String
```

Reason:

```
Every character is a letter or digit.
```

26. Test 2: Invalid special character

Input:

```
An?1
```

Expected output:

```
Invalid String
```

Reason:

? is not allowed.

27. Test 3: Underscore with original rule

Input:

user_name

Expected output with original rule:

Invalid String

Reason:

_ is not a letter and not a digit.

Expected output if underscore is allowed:

Valid String

28. Test 4: All digits

Input:

123456

Expected output:

Valid String

Reason:

Digits are allowed.

29. Test 5: Invalid character at the end

Input:

```
User123!
```

Expected output:

```
Invalid String
```

Reason:

```
! is not allowed.
```

This test is important because it proves the loop checks the full string before returning `1`.

30. Test 6: Empty string

Input:

```
""
```

Memory:

```
\0
```

The loop does not run because the first character is already `\0`.

With the current function, the result is:

```
Valid String
```

Why?

Because no invalid character was found.

But in real applications, you may want to reject empty strings.

To reject empty strings, add this check:

```
if (name[0] == '\0') {  
    return 0;  
}
```

Then continue with the normal validation loop.

Part J: Complexity analysis

31. Time complexity

The function scans the string from left to right.

If the string has N characters, then in the worst case it checks all N characters.

Worst-case time complexity:

$O(N)$

When does the worst case happen?

When the string is valid, or when the invalid character is at the end.

Example:

User123!

The function checks almost the whole string before finding **!**.

32. Best-case time complexity

Best case happens when the first character is invalid.

Example:

?Anil

The function sees `?` immediately and returns `0`.

Best-case time:

`O(1)`

But when we usually describe the algorithm, we use worst-case time:

`O(N)`

33. Space complexity

The function uses only a few variables:

```
int i;
```

or in the cleaner version:

```
int i;  
int isUpper;  
int isLower;  
int isDigit;
```

It does not create another array.

So extra space complexity is:

`O(1)`

This means constant extra space.

34. Complexity table

Operation	Time Complexity	Extra Space
Validate a string	<code>O(N)</code> worst case	<code>O(1)</code>
Reject first invalid character	<code>O(1)</code> best case	<code>O(1)</code>

Part K: Common mistakes

35. Mistake 1: Returning valid too early

Wrong idea:

If I find one valid character, the whole string is valid.

This is incorrect.

Example:

A???

A is valid, but the full string is invalid.

Correct idea:

The string is valid only if every character is valid.

36. Mistake 2: Forgetting digits

Wrong validation:

```
if (!(name[i] >= 'A' && name[i] <= 'Z') &&
    !(name[i] >= 'a' && name[i] <= 'z')) {
    return 0;
}
```

This rejects digits.

So this would wrongly reject:

Anil2321

Correct validation must include:

```
name[i] >= '0' && name[i] <= '9'
```

37. Mistake 3: Using the wrong logical operator

This is wrong:

```
if (!(name[i] >= 'A' && name[i] <= 'Z') ||
    !(name[i] >= 'a' && name[i] <= 'z') ||
    !(name[i] >= '0' && name[i] <= '9')) {
    return 0;
}
```

Why?

Because it uses `||`, meaning OR.

A character cannot be uppercase, lowercase, and digit at the same time.

So this condition would reject almost everything.

Correct version uses `&&`:

```
if (!isUpper && !isLower && !isDigit) {
    return 0;
}
```

Meaning:

Reject only if it fails all allowed checks.

38. Mistake 4: Not stopping at `\0`

Wrong:


```
for (i = 0; i < 100; i++) {  
    // check name[i]  
}
```

Better:

```
for (i = 0; name[i] != '\0'; i++) {  
    // check name[i]  
}
```

Why?

Because the string may end before index `100`.

The real end of a C-style string is marked by `\0`.

39. Mistake 5: Thinking symbols are allowed automatically

Symbols like these are not allowed by the original rule:

```
_ ? ! @ # space
```

If you want to allow a symbol, you must add it to the condition.

Example:

```
name[i] == '_'
```

Part L: Main code patterns to remember

40. Pattern 1: Check uppercase

```
name[i] >= 'A' && name[i] <= 'Z'
```

41. Pattern 2: Check lowercase

```
name[i] >= 'a' && name[i] <= 'z'
```

42. Pattern 3: Check digit

```
name[i] >= '0' && name[i] <= '9'
```

43. Pattern 4: Reject if none of the allowed checks pass

```
if (!(name[i] >= 'A' && name[i] <= 'Z') &&  
    !(name[i] >= 'a' && name[i] <= 'z') &&  
    !(name[i] >= '0' && name[i] <= '9')) {  
    return 0;  
}
```

44. Pattern 5: Full validation function

```
int validate(char *name) {  
    int i;  
  
    for (i = 0; name[i] != '\0'; i++) {  
        if (!(name[i] >= 'A' && name[i] <= 'Z') &&  
            !(name[i] >= 'a' && name[i] <= 'z') &&  
            !(name[i] >= '0' && name[i] <= '9')) {  
            return 0;  
        }  
    }  
  
    return 1;  
}
```

Part M: Full combined example

45. Program to validate a string

```
#include <stdio.h>

int validate(char *name) {
    int i;

    for (i = 0; name[i] != '\0'; i++) {
        if (!(name[i] >= 'A' && name[i] <= 'Z') &&
            !(name[i] >= 'a' && name[i] <= 'z') &&
            !(name[i] >= '0' && name[i] <= '9')) {
            return 0;
        }
    }

    return 1;
}

int main() {
    char name[] = "An?1";

    if (validate(name)) {
        printf("Valid String\n");
    } else {
        printf("Invalid String\n");
    }

    return 0;
}
```

Output:

```
Invalid String
```

Part N: Practice questions

46. Practice 1

Validate:

Anil2321

Expected answer:

Valid String

Reason:

All characters are letters or digits.

47. Practice 2

Validate:

An?1

Expected answer:

Invalid String

Reason:

? is not allowed.

48. Practice 3

Validate with the original rule:

user_name

Expected answer:

Invalid String

Reason:

```
_ is not allowed by the original rule.
```

49. Practice 4

Modify the validation condition to allow underscore.

Hint:

Add this safe condition:

```
name[i] == '_'
```

50. Practice 5

Modify the validation condition to allow spaces.

Hint:

Add this safe condition:

```
name[i] == ' '
```

Then explain why spaces may still be rejected in usernames.

51. Practice 6

Test these strings:

```
""  
"12345"  
"abcDEF99"  
"abc!"  
"?abc"  
"abc?"
```

Questions:

1. Which ones are valid?
 2. Which ones are invalid?
 3. Which one stops immediately?
 4. Which one forces the loop to scan until the end before failing?
-

Part O: Final summary

52. Key points to remember

1. Validation means checking whether a string follows a rule.
 2. In Module 4, the rule allows only letters and numbers.
 3. Uppercase letters are from 'A' to 'Z'.
 4. Lowercase letters are from 'a' to 'z'.
 5. Digits are from '0' to '9'.
 6. Symbols like '?', '!', '_', and spaces are invalid unless we add them to the rule.
 7. The function scans the string from left to right.
 8. The loop stops at '\0'.
 9. If an invalid character is found, return 0 immediately.
 10. Return 1 only after the whole string has passed.
 11. Worst-case time complexity is $O(N)$.
 12. Extra space complexity is $O(1)$.
-

53. Final mental model

Think of validation like checking people at a gate.

Each character tries to enter:

```
A -> allowed  
n -> allowed  
? -> rejected
```

The moment one character is rejected, the whole string is invalid.

So the main idea is:

```
Check each character.  
If it belongs to an allowed range, continue.
```

If it does not belong to any allowed range, stop and return invalid.
If the whole string finishes without problems, return valid.

Final takeaway:

Module 4 is about scanning a string and making sure every character is allowed. If even one character is outside the allowed ranges, the string is invalid.